

Final Project

Smart Insect Identifier & AI Insights

1. Deskripsi Proyek

Praktikan diwajibkan membangun sistem identifikasi serangga otomatis yang mampu mengenali spesies serangga dari gambar dan memberikan wawasan mendalam (taksonomi, habitat, dan *fun facts*) menggunakan integrasi Large Language Model (LLM) Gemini.

2. Ketentuan Umum

- **Dataset:** Gunakan dataset citra serangga yang telah disediakan di [Kaggle](#).
- **LLM:** Gunakan LLM dari Gemini's API yang telah disediakan di [Google AI Studio](#).
- **Download Project:** Clone project dari [Github](#).
- **Framework ML:** Bebas menggunakan **PyTorch** atau **TensorFlow/Keras**, atau framework lain yang relevan.
- **Framework Frontend & Backend:** Bebas menggunakan **Next.js & Tailwind CSS**, **Gradio** (sebagai frontend) dan **FastAPI** (sebagai backend) atau framework lain yang relevan. (**Catatan: kode, arsitektur, dan framework hanya berupa template/contoh saja, silahkan di ubah, tambah, atau kurangi sesuai kebutuhan!**)
- **UI:** Jika praktikan menggunakan tampilan yang telah disediakan, maka praktikan diwajibkan melakukan minimal beberapa penyesuaian pada antarmuka, seperti mengganti warna, tipografi, ikon, atau elemen visual lainnya. Tampilan akhir tidak diperbolehkan sama persis dengan template asli, sehingga tetap menunjukkan adanya proses pengembangan dan kustomisasi oleh mahasiswa.
- **Arsitektur:** Praktikan boleh menggunakan teknik *Transfer Learning* (contoh: EfficientNet, ResNet, atau MobileNet), ataupun melatih model dari awal.
- **Komputasi:** Praktikan sangat disarankan menggunakan GPU untuk melatih model seperti Kaggle, ataupun Collab.

3. Alur Kerja & Persyaratan Teknis

Berikut adalah struktur direktori yang akan kita bangun:

```
LensArthropoda/  
├── files/  
│   ├── laporan.pdf  
│   └── notebook.ipynb  
├── backend/  
│   ├── artifacts/  
│   │   └── model_yang_disimpan  
│   ├── env.example  
│   ├── gemini_service.py  
│   ├── main.py  
│   ├── ml_service.py  
│   └── requirements.txt  
└── frontend/  
    ├── app/  
    │   ├── page.tsx  
    │   ├── globals.css  
    │   └── layout.tsx  
    ├── components/  
    │   └── ResultCard.tsx  
    ├── package.json  
    ├── ....  
    ├── ....  
    └── tsconfig.json
```

Gambar 3.1 Struktur Direktori Project

Struktur yang diberikan adalah sebagai template project sehingga praktikan bebas mengubah atau mengganti sesuai dengan framework yang digunakan.

A. Tahap Pelatihan Model (Training)

- **Preprocessing:** Lakukan preprocessing yang sesuai seperti resize image, normalisasi, augmentasi data, encoding label, serta penyesuaian input berdasarkan arsitektur pretrained model yang digunakan. (**NOTE:** *Pastikan melakukan penanganan gambar yang rusak pada dataset agar proses training dapat berjalan dengan benar*).
- **Training:** Lakukan proses training dengan menggunakan early stopping untuk mencegah overfitting.
- **Evaluasi:** Lakukan proses evaluasi model dengan menampilkan accuracy, classification report, confusion matrix, training-validation curve untuk loss dan akurasi.
- **Model Export:** Model dapat diekspor ke format yang efisien untuk *production*:
 - **PyTorch:** Misalnya `torch.jit.script` atau `torch.jit.save`.
 - **TensorFlow:** Seperti format `.h5` atau `SavedModel`.

B. Tahap Pengembangan Backend (API)

Membangun REST API menggunakan **FastAPI** yang melayani:

- **Inference:** Memuat model yang telah dilatih dan memproses gambar kiriman *user*.
- **Gemini Integration:** Mengintegrasikan **Gemini 2.5 Flash Lite** dengan fitur:
 - `ThinkingConfig`
 - **Opsional:** `GoogleSearch` (Grounding untuk mendapatkan data dari hasil searching).
- **Fallback Mechanism:** API harus tetap memberikan hasil prediksi model meskipun koneksi API Gemini sedang sibuk/limit (Status 503).

Di dalam direktori backend terdapat beberapa file seperti berikut:



Gambar 3.2 Struktur Direktori Backend

Direktori `artifacts` digunakan untuk menyimpan model yang telah di training dan metadata model. File `env.example` adalah file template yang digunakan untuk menyimpan Gemini API KEY yang akan digunakan pada project. File `gemini_service.py` adalah file yang digunakan untuk mengatur pemanggilan API Google Gemini. File `main.py` berfungsi sebagai entry point atau server utama backend API yang menggunakan framework FastAPI.

File `ml_service.py` adalah kelas yang digunakan untuk mengklasifikasi gambar. File `requirements.txt` berisi daftar dependency/library yang dibutuhkan aplikasi agar bisa berjalan.

Praktikan diharuskan mengikuti langkah-langkah berikut agar backend dapat berjalan dengan benar:

1. Salin file `env.example` menjadi file `.env`, kemudian isi variabel API menggunakan kredensial yang diperoleh dari [Google AI Studio](#).
2. Lengkapi file `ml_service.py` sesuai alur dan konfigurasi yang digunakan saat proses training model, agar proses inference dapat berjalan secara konsisten dan menghasilkan output yang sesuai.
3. Lengkapi implementasi pada file `gemini_service.py` untuk menangani integrasi layanan Gemini API.
4. Lengkapi file `main.py` sebagai entry point aplikasi backend. Pastikan seluruh routing, dependency, dan inisialisasi service telah terhubung dengan benar.
5. Tambahkan seluruh dependency yang dibutuhkan ke dalam file `requirements.txt`.
6. Setelah itu, lakukan instalasi dependency menggunakan `pip` agar seluruh modul yang diperlukan dapat digunakan sistem. Disarankan menjalankan instalasi dependency di dalam virtual environment (`venv`) agar dependensi proyek tetap terisolasi dan tidak bertabrakan dengan konfigurasi global Python.

Contoh penggunaan virtual environment:

```
python -m venv env
```

Aktivasi environment:

Linux/macOS:

```
source env/bin/activate
```

Windows:

```
env\Scripts\activate
```

7. Setelah virtual environment aktif, install dependency:

```
pip install -r requirements.txt
```

8. Untuk menjalankan backend gunakan perintah berikut:

```
uvicorn main:app --reload
```

Pastikan backend berhasil berjalan tanpa error dan endpoint mampu mengembalikan response/output dengan benar.

C. Tahap Front End (UI/UX)

Antarmuka dibangun agar dapat berinteraksi dengan fitur:

- **Image Upload & Preview.**
- **Markdown Rendering:** Menggunakan `react-markdown` untuk menampilkan hasil dari Gemini secara rapi.

- **Visual Feedback:** Tampilan *loading state* saat AI sedang "berpikir".

Berikut adalah struktur direktori frontend yang diberikan:



Gambar 3.3 Struktur Direktori Frontend

Bagian frontend praktikan dibebaskan untuk membangun tampilan seperti yang diinginkan. Kode yang diberikan hanya sebagai template saja.

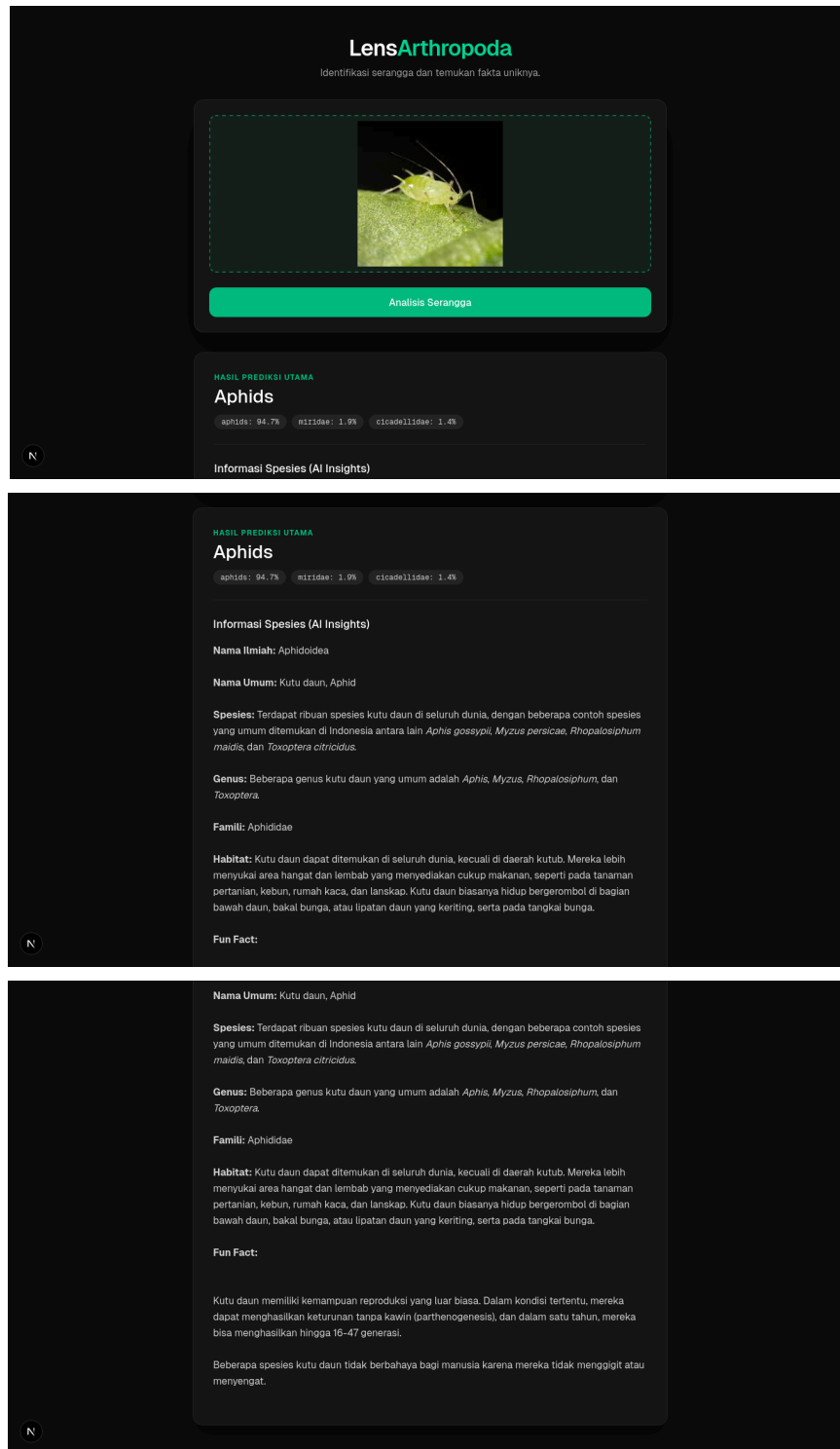
Berikut langkah menjalankan frontend apabila menggunakan template yang sudah ada:

1. Pastikan Node.js dan npm sudah terinstall pada sistem.
2. Install seluruh dependency project dengan perintah berikut:
`npm install`
3. Setelah proses instalasi selesai, jalankan frontend menggunakan perintah:
`npm run dev`
4. Pastikan aplikasi frontend berhasil berjalan tanpa error dan dapat diakses melalui browser pada alamat yang ditampilkan di terminal.

6. Deliverables (Project Output)

1. **Source code** berisi semua code yang digunakan.
2. **Laporan (PDF)** yang menjelaskan:
 - Evaluasi model.
 - Saran pengembangan model lanjutan.
 - Tangkapan layar (Screenshot) aplikasi saat berhasil melakukan prediksi.
 - Analisis singkat mengenai jawaban Gemini terhadap hasil prediksi model.
3. **Video** 5-8 menit demo project, dan penjelasan arsitektur, framework yang digunakan.
4. **Post showcase** hasil project yang telah dibuat pada linkedin dengan menetag Aslab [Agil](#) & [Willy](#).

Contoh tampilan hasil project:



7. Submission (GCR)

1. Kumpulkan **link Github** proyek yang telah selesai,
 - a. Pastikan file `.env` dan folder virtual environment tidak di upload ke github
2. Kumpulkan **link drive atau Youtube** video demo presentasi,
3. **Laporan** yang telah dibuat

7. Kriteria Penilaian

Kriteria	Bobot	Detail
Notebook	20%	Tahap preprocessing data, kerapian, kejelasan, dan evaluasi dari proses pembangunan model yang dibuat.
Model Accuracy	10%	Performa model dalam mengenali serangga pada data uji.
Backend Logic	15%	Keberhasilan integrasi API, error handling, dan modularitas.
Frontend UI	10%	Desain yang bersih, indah, dan nyaman dilihat.
Gemini Integration	10%	Efektivitas prompting dan penggunaan fitur Google Search.
Laporan	15%	Kelengkapan laporan sesuai dengan bagian deliverables yang diminta.
Video Demonstrasi	20%	Demonstrasi dari project.

8. FAQ

1. Q: Apakah saya boleh menggunakan dataset tambahan dari luar?

A: Diperbolehkan, selama dataset utama yang diberikan tetap digunakan, sumber data dicantumkan, dan berasal dari sumber yang jelas. Penambahan data (Data Augmentation) atau penggunaan dataset eksternal yang relevan sangat disarankan untuk meningkatkan akurasi dan ketahanan model terhadap variasi gambar.

2. Q: Apakah saya boleh tidak menggunakan keseluruhan dataset karena ukurannya sangat besar?

A: Diperbolehkan, selama dapat dipastikan model tidak menjadi underfitting, dan menyertakan alasannya di laporan. Sangat disarankan menggunakan *cloud server* (notebook via kaggle/Collab) sehingga tidak memberatkan *storage* untuk mendownload dataset dan pelatihan model (kami sudah membuktikan untuk dataset ini).

3. Q: Bagaimana jika laptop saya tidak memiliki GPU atau setup CUDA untuk menjalankan Backend?

A: Proses *training* sangat disarankan menggunakan GPU (via Kaggle/Colab). Namun, untuk tahap *deployment* di Backend lokal, Anda wajib memastikan model di-load ke

CPU menggunakan parameter `map_location='cpu'` pada PyTorch atau konfigurasi serupa di framework lain agar tidak terjadi *runtime error*.

4. Q: Di mana saya bisa mendapatkan API Key Gemini dan apakah berbayar?

A: Anda bisa mendapatkan API Key secara gratis melalui [Google AI Studio](#). Pastikan Anda memilih paket "Free Tier" yang tersedia untuk keperluan pengembangan dan eksperimen, dan disarankan untuk menggunakan email usk.

5. Q: Bagaimana saya mengetahui rate limit API Key Gemini untuk Free Tier?

A: Baca Panduan [Rate Limits](#) untuk melihat batasan *free tier* Gemini Flash 2.5 Lite.

6. Q: Mengapa saya harus menggunakan file `.env` untuk menyimpan API Key?

A: Ini adalah praktik *Clean Code* dan keamanan dasar. Menyimpan API Key langsung di dalam kode (*hard coding*) sangat berbahaya jika repository Anda diunggah ke GitHub publik. Gunakan library `python-dotenv` untuk membaca key secara aman.

7. Q: Mengapa saya harus menggunakan file `.gitignore` dan apa saja yang perlu dimasukkan di dalamnya?

A: Penggunaan `.gitignore` adalah praktik terbaik untuk manajemen repositori, untuk alasan keamanan dan efisiensi.

- a. **Keamanan:** Wajib untuk mencegah file sensitif seperti `.env` terunggah ke GitHub publik.
- b. **Efisiensi:** Mencegah *commit* file besar yang bisa diregenerasi, seperti folder *virtual environment* Python (misalnya `env/` atau `venv/`) dan *dependency* Node.js (`node_modules/`). Folder-folder ini dapat direstorasi dengan `pip install -r requirements.txt` atau `npm install`.

8. Q: Apa yang harus saya lakukan jika Gemini API terus memberikan error 503 (Service Unavailable)?

A: Error 503 biasanya terjadi karena lonjakan trafik pada server Google. Mahasiswa bisa mengimplementasikan *Try-Except block* di backend. Jika Gemini gagal merespons, sistem dapat tetap menampilkan nama serangga hasil prediksi model ML Anda dengan pesan tambahan bahwa informasi detail AI sedang tidak tersedia.

9. Q: Apakah tampilan UI harus sama persis dengan contoh?

A: Tidak. Contoh yang diberikan hanyalah standar minimum. Mahasiswa dibebaskan bahkan didorong untuk berinovasi pada UI/UX, misalnya dengan menambahkan fitur riwayat prediksi (*history*), mode gelap/terang, atau visualisasi probabilitas yang lebih interaktif menggunakan grafik.

10. Q: Mengapa hasil teks dari Gemini terlihat berantakan dengan simbol bintang-bintang?

A: Itu adalah format Markdown. Pastikan di bagian Frontend Anda sudah mengimplementasikan library markdown seperti `react-markdown` ataupun plugin `@tailwindcss/typography` agar simbol tersebut diterjemahkan menjadi teks tebal, list, dan heading yang rapi.